Cybersecurity by Design for Critical Infrastructures

Laurea Magistrale in Informatica

alma.pennacchi@uniroma2.it

Marzo 2026

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Roadmap

1 Detection Methods

Lezione 3 — Obiettivo

Domanda guida

Come si costruisce un sistema di detection utile, robusto e difendibile in una infrastruttura critica?

- trasformare osservazioni in **decisioni operative**
- capire come si calcolano **deviazioni**, **score** e **soglie**
- confrontare famiglie diverse di detector

La detection pipeline

Formalizzazione

$$\mathcal{O} \to \Phi(\mathcal{O}) \to S \to D$$

dove:

- $\mathcal{O}$ = osservabili
- $\Phi(\mathcal{O})$ = feature estratte
- S = score o segnali di deviazione
- D = decisione operativa

Osservabili vs Feature

Osservabili

- flussi
- log
- retry
- latenze
- comandi

Feature

- tasso di retry
- volume medio per finestra
- frequenza connessioni
- numero di destinazioni
- ordine degli eventi

Punto chiave

Un detector ragiona quasi sempre su **feature**, non sul dato grezzo.

Score e soglie

Idea

Un detector calcola uno **score di deviazione** e lo confronta con una soglia.

$$x \to S(x) \to \text{alert se } S(x) > \tau$$

- soglia troppo bassa $\Rightarrow$ rumore
- soglia troppo alta $\Rightarrow$ eventi persi

Come si calcola una deviazione?

Idea

Una deviazione è una misura della distanza tra il comportamento osservato e il comportamento atteso.

$$\text{deviazione} = d(x, B)$$

dove:

- x = osservazione corrente
- B = baseline o regione attesa
- d = misura di distanza

Deviazione assoluta

Il caso più semplice:

$$d(x) = |x - \mu|$$

dove:

- x = valore osservato
- μ = valore atteso

Interpretazione

Misura di quanto l'osservazione si discosta dal centro della baseline.

Deviazione relativa

A volte conta la deviazione in percentuale:

$$d_{rel}(x) = \frac{|x - \mu|}{\mu}$$

Quando è utile

Quando la scala dei valori cambia molto tra asset, fasce temporali o modalità operative.

Esempio concettuale

Uno scostamento di 5 unità non ha lo stesso significato se il valore atteso è 10 oppure 1000.

Deviazione standardizzata: z-score

Una deviazione non va letta solo in valore assoluto, ma rispetto alla variabilità normale.

$$z = \frac{x - \mu}{\sigma}$$

dove:

- μ = media attesa
- σ = deviazione standard

Interpretazione

Lo stesso scostamento può essere irrilevante o gravissimo, a seconda di quanto il sistema oscilla normalmente.

Stessa deviazione, significato diverso

Caso A:

$$\mu = 9, \quad \sigma = 0.5, \quad x = 15 \Rightarrow z = 12$$

Caso B:

$$\mu = 9, \quad \sigma = 5, \quad x = 15 \Rightarrow z = 1.2$$

Punto chiave

Il significato dell'osservazione dipende dalla dispersione attesa, non solo dal suo valore assoluto.

Baseline come intervallo

In molti casi la baseline può essere modellata come intervallo ammesso:

$$x \in [L, U]$$

Regola semplice

Se osserviamo $x < L$ oppure $x > U$, segnaliamo deviazione.

Interpretazione

La baseline può essere vista come un inviluppo operativo, non necessariamente come un singolo punto medio.

Hard threshold vs Soft threshold

Hard threshold

$$x > \tau \Rightarrow alert$$

Decisione netta.

Soft threshold

$$S(x) = f(x)$$

Più cresce la deviazione, più cresce lo score.

Conseguenza

Nei sistemi reali spesso conviene usare uno **score continuo**, non solo una soglia binaria.

Costruire uno score semplice

Un modo elementare:

$$S(x) = \max\left(0, \frac{x - U}{U - L}\right)$$

dove:

- $[L, U]$ = intervallo normale
- $S(x) = 0$ se il valore è nella baseline
- $S(x) > 0$ se il valore eccede il limite

Interpretazione

Lo score misura di quanto l'osservazione esce dalla fascia attesa.

Score come funzione esplicita

Supponiamo di osservare:

- packet rate
- retry rate
- latency

Per ogni feature calcoliamo una deviazione normalizzata:

$$s_i = \frac{x_i - \mu_i}{\sigma_i}$$

Poi combiniamo:

$$S = w_1 s_{pkt} + w_2 s_{retry} + w_3 s_{lat}$$

Messaggio

Lo score non si assegna "a mano": si calcola tramite una funzione esplicita.

Deviation Score vs Severity Score

Deviation Score

Misura quanto il comportamento osservato si discosta dalla baseline.

Severity Score

Misura quanto quella deviazione è pericolosa nel contesto operativo.

$$\text{Severity} = g(\text{Deviation}, \text{Criticality}, \text{Context})$$

Punto chiave

Una deviazione alta non coincide sempre con una severità alta, e viceversa.

Score multi-feature

Spesso una sola misura non basta.
Possiamo combinare più deviazioni:

$$S = \alpha S_{latency} + \beta S_{retry} + \gamma S_{volume}$$

- latenza
- retry
- volume
- numero di destinazioni

Idea

La deviazione complessiva può emergere da più segnali deboli che, isolati, non bastano.

Soglie robuste

Media e deviazione standard funzionano bene se i dati sono puliti e abbastanza regolari. Quando ci sono outlier, si preferiscono misure robuste:

- **mediana**
- **MAD** (Median Absolute Deviation)

$$MAD = \text{mediana}(|x_i - \text{mediana}(x)|)$$

Perché

Gli outlier non devono definire da soli la baseline.

Come si fissano le soglie

Tre approcci principali:

1. **statistical**: es. $|z| > 3$
2. **percentile-based**: es. oltre il 95% o 99% dei valori normali
3. **risk-based**: soglia scelta in base al costo dell'errore

Idea

La soglia non è arbitraria se deriva da una regola esplicita e giustificabile.

Come si scelgono i pesi

I pesi in uno score combinato non dovrebbero essere scelti "a sensazione".
Approcci tipici:

- **expert-driven**: conoscenza di dominio
- **data-driven**: apprendimento da storico etichettato
- **risk-driven**: riflettono il costo operativo

Messaggio

L'oggettività non sta nell'assenza di scelte, ma nell'esplicitare e giustificare le scelte.

Cosa rende uno score difendibile

Uno score è accettabile se è:

- **esplicito**
- **riproducibile**
- **calcolabile**
- **motivato**
- **coerente con il contesto**

Idea

Due analisti diversi devono ottenere lo stesso risultato se applicano la stessa funzione agli stessi dati.

Famiglie di detector

Quattro famiglie principali:

1. **rule-based**
2. **statistical**
3. **sequential / temporal**
4. **graph-based**

Messaggio

In sistemi reali nessuna famiglia è sufficiente da sola.

Rule-Based Detection

Definizione

Un detector rule-based segnala una deviazione quando una regola esplicita viene violata.

- allowlist topologica
- denylist
- policy violation
- signature matching

Esempi di regole

Esempi tipici:

- `GuestWiFi` $\rightarrow$ `EHR` non ammesso
- `PLC` $\rightarrow$ `Internet` non ammesso
- comando `write` fuori finestra operativa
- accesso remoto senza MFA

Vantaggio

Le regole sono leggibili, auditabili e facilmente difendibili.

Vantaggi e limiti delle regole

Vantaggi

- interpretabili
- spiegabili
- adatte a policy e compliance

Limiti

- coprono ciò che già conosciamo
- rigide rispetto al contesto
- poco adatte a novità non previste

Statistical Detection

Idea

Un detector statistico segnala scostamenti quantitativi rispetto a una distribuzione attesa.

- media e deviazione standard
- soglie dinamiche
- percentili
- sliding windows

Esempio: z-score

$$z = \frac{x - \mu}{\sigma}$$

- $|z|$ piccolo $\Rightarrow$ comportamento vicino alla media
- $|z|$ grande $\Rightarrow$ scostamento significativo

Attenzione

La statistica misura differenze numeriche, ma non comprende da sola il significato operativo del sistema.

Sliding Windows e aggregazione

La detection statistica spesso lavora per finestre temporali:

- pacchetti per minuto
- retry per finestra
- numero di destinazioni uniche
- volume medio per fascia oraria

Perché

Una singola osservazione può essere irrilevante; una sequenza aggregata può diventare significativa.

Limiti della pura statistica

- fatica con distribuzioni multimodali
- può ignorare la topologia
- può perdere il significato della sequenza
- tende a produrre alert poco spiegabili senza contesto

Idea

Un picco di traffico può essere sospetto o perfettamente lecito: dipende da **chi**, **quando**, **dove** e **perché**.

Sequential / Temporal Detection

Definizione

La detection temporale verifica se eventi e comandi avvengono in un ordine compatibile con il processo atteso.

- ordine degli eventi
- intervalli temporali
- transizioni di stato
- burst e concatenazioni anomale

Perché la sequenza conta

Esempi:

- login $\rightarrow$ accesso $\rightarrow$ export
- read $\rightarrow$ validation $\rightarrow$ write
- update $\rightarrow$ restart $\rightarrow$ sync

Punto chiave

Due eventi leciti presi separatamente possono essere anomali se appaiono in **ordine sbagliato**.

State-Transition Detection

Possiamo modellare il sistema come una macchina a stati.

$$\text{idle} \rightarrow \text{active} \rightarrow \text{maintenance} \rightarrow \text{recovery}$$

- alcune transizioni sono normali
- altre sono rare
- altre non dovrebbero esistere

Graph-Based Detection

Idea

Una deviazione può essere strutturale: non nel volume, ma nella forma del grafo osservato.

- nuovo nodo
- nuovo edge
- edge atteso che scompare
- cambiamento di ruolo topologico

Anomalie strutturali

Esempi:

- comparsa di `DB` $\rightarrow$ `UnknownHost`
- dispositivo medico che parla direttamente con il database
- nodo guest che acquisisce centralità inattesa
- nuovo path che attraversa zone non previste

Punto forte

Il detector grafico vede pattern che i metodi solo statistici possono non vedere.

Centrality Shift

Anche senza nuovi nodi, un asset può cambiare ruolo.

- aumento di degree
- aumento di betweenness
- nuovo ruolo di pivot
- propagazione più probabile del rischio

Interpretazione

Un cambiamento di centralità può segnalare compromissione, errore di configurazione o riorganizzazione non censita.

Hybrid Detection

Idea

I sistemi reali combinano più detector.

$$S = \alpha S_{rule} + \beta S_{stat} + \gamma S_{seq} + \delta S_{graph}$$

- regole per interpretabilità
- statistica per sensibilità
- sequenze per contesto
- grafi per struttura

Perché servono detector ibridi

- una regola può vedere una violazione topologica ma non un drift graduale
- la statistica può vedere un picco ma non capire se il path era ammesso
- la sequenza può vedere ordine anomalo ma non un nuovo nodo
- il grafo può vedere un nuovo edge ma non un burst temporale fine

Limiti operativi dei detector

Problemi reali:

- dati incompleti
- drift della baseline
- tuning difficile
- alert fatigue
- costo di manutenzione
- mismatch tra modello e realtà operativa

Explainability

Domanda

Un operatore può spiegare perché l'allarme è stato generato?

Un alert utile deve essere:

- interpretabile
- giustificabile
- riproducibile
- collegabile a una contromisura

Detection utile vs detection teorica

Messaggio

Un detector eccellente in laboratorio può essere inutilizzabile in produzione.

- troppi falsi positivi
- manutenzione troppo costosa
- dipendenza da dati che in realtà non esistono
- impossibilità di spiegare l'allarme

Messaggio finale della teoria

Idea centrale

La detection efficace in infrastrutture critiche non dipende da un singolo algoritmo, ma dalla progettazione di detector **coerenti con il contesto operativo**.